

CS 3960 (3)

Two Visions for Future SWE

Spring 2026, *content* Pavel Panchekha and John Regehr, *illustrations* Gemini 3 Nano Banana Pro & 2

Projects

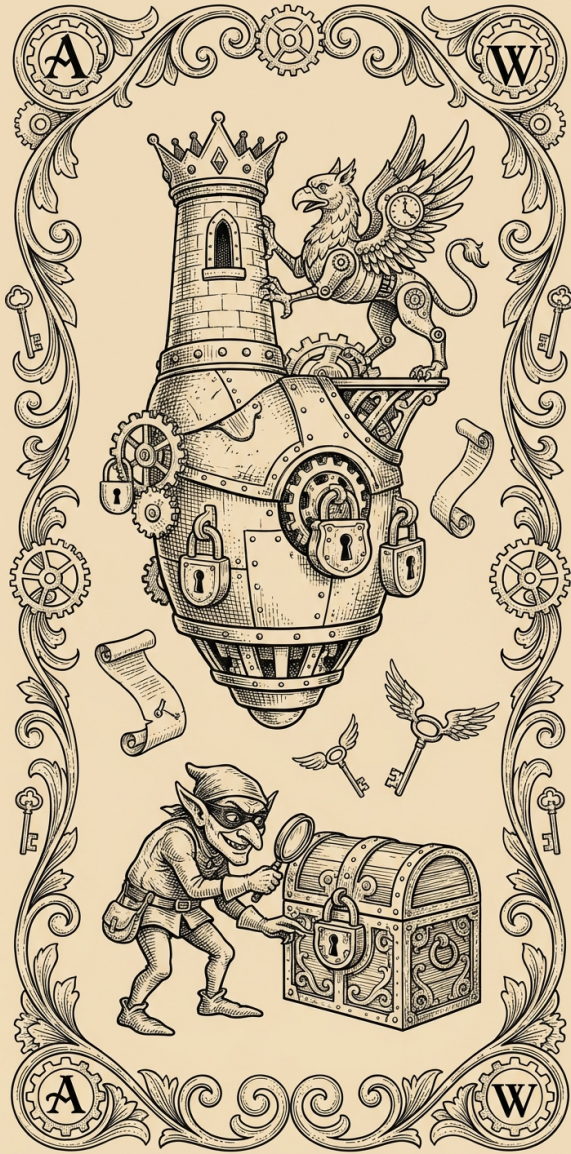
Any last questions for us?

Reminder:

- Your video is due in two days
- Last handin time: Sunday night

Today: LLMs and Computer Security

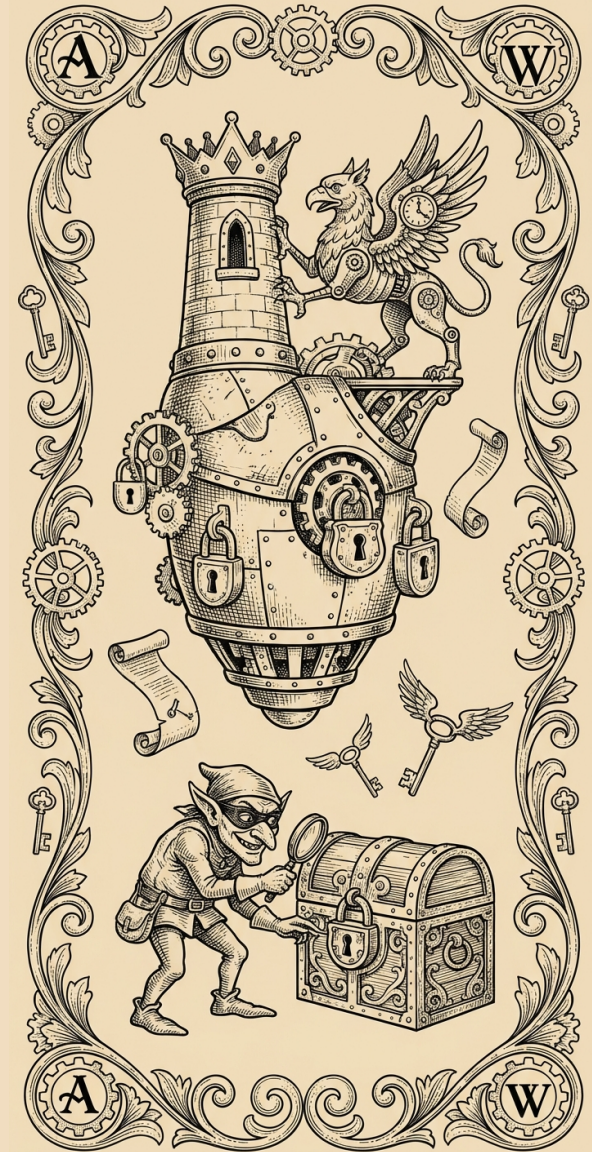
What's even going on here?

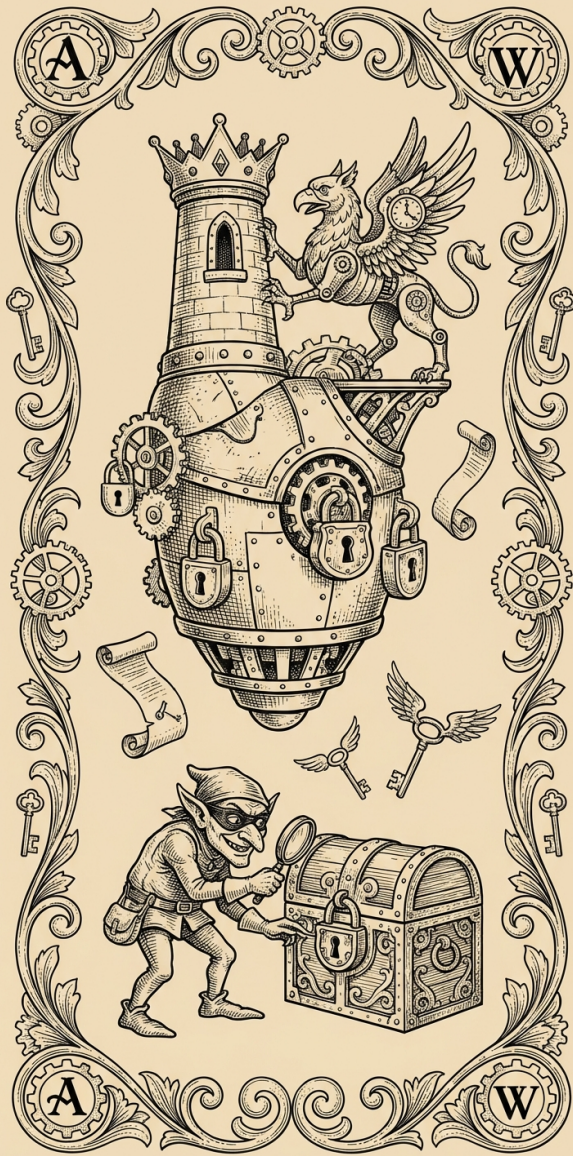


A computer system is insecure if it allows someone to do something that they shouldn't be able to do

What everyone should and shouldn't be able to do is the **security policy**

Alas, it's often the case that nobody actually articulates the security policy





Policy examples:

User A shouldn't be able to read memory belonging to User B

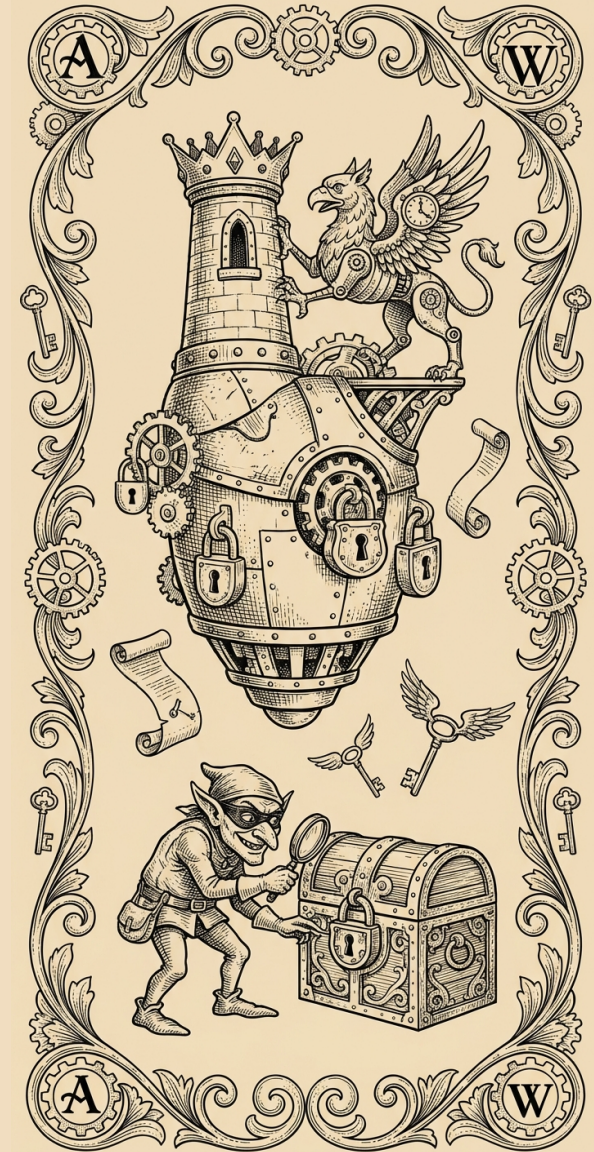
Classified documents should never be read by users lacking the proper clearance

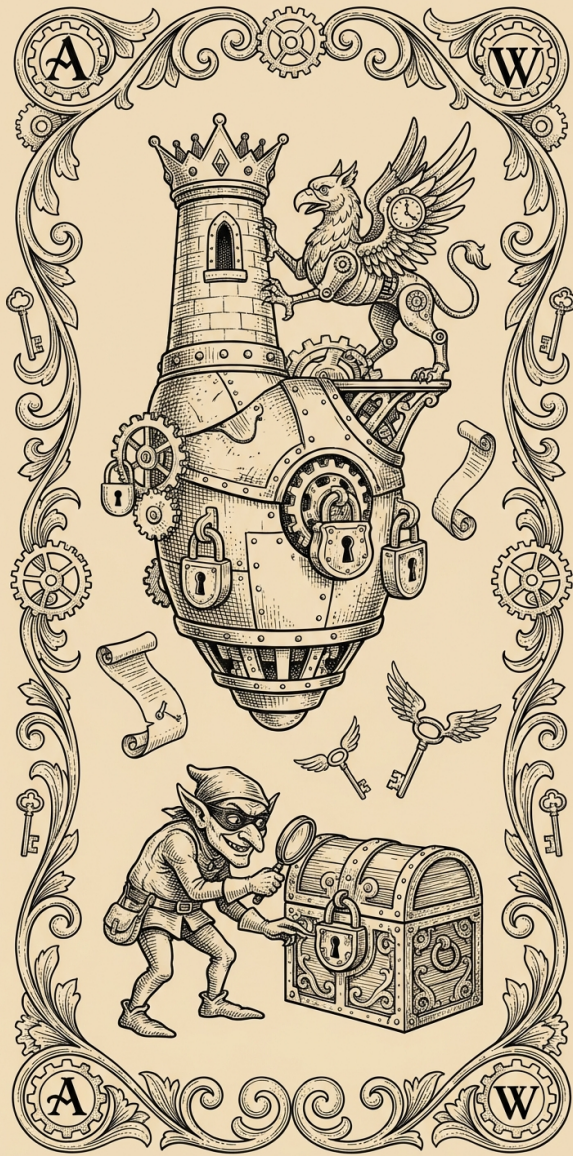
Only users with the proper access tokens should be able to log into the system

There a lot of variation here. Security policies can be...

- formal or informal
- high-level or low-level
- written down, or simply understood by stakeholders

Also, security policies tend to drift over time. For example “the internet”





A lot of projects never actually write down their security policy

Even huge ones like Linux, Adobe Acrobat, Google Chrome

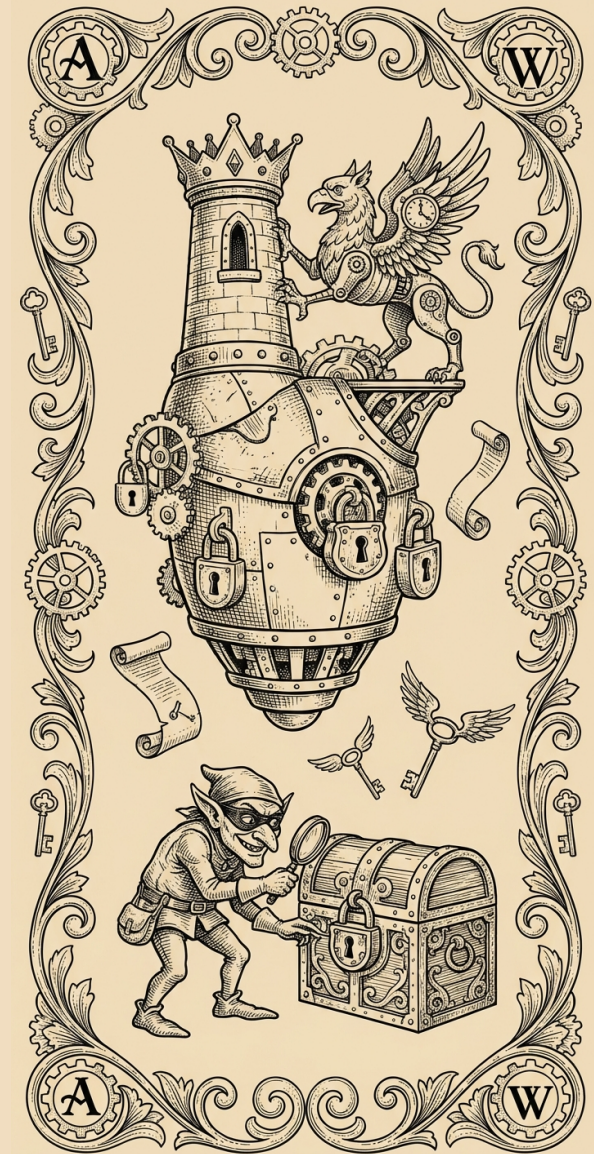
This is sort of OK in practice because the humans collectively understand what to do / not to do

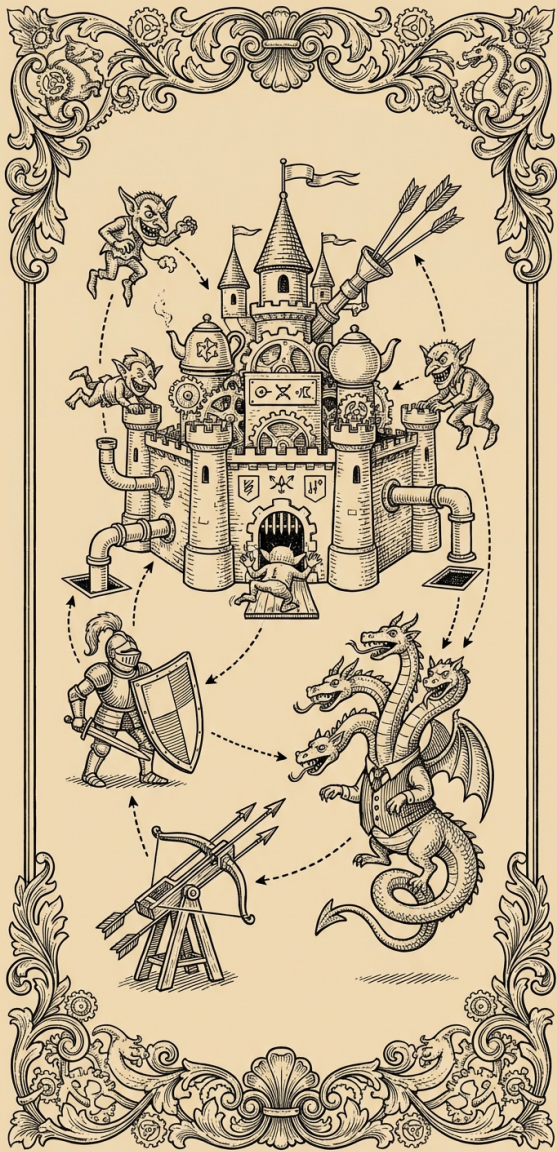
But what about the LLMs?

We could write down a security policy and
ask the LLM to respect it

This is worth trying

But don't believe for a second that this is
what will happen





To talk about security we also want a **threat model**, which answers questions like:

Who's attacking our system?

What are they trying to accomplish?

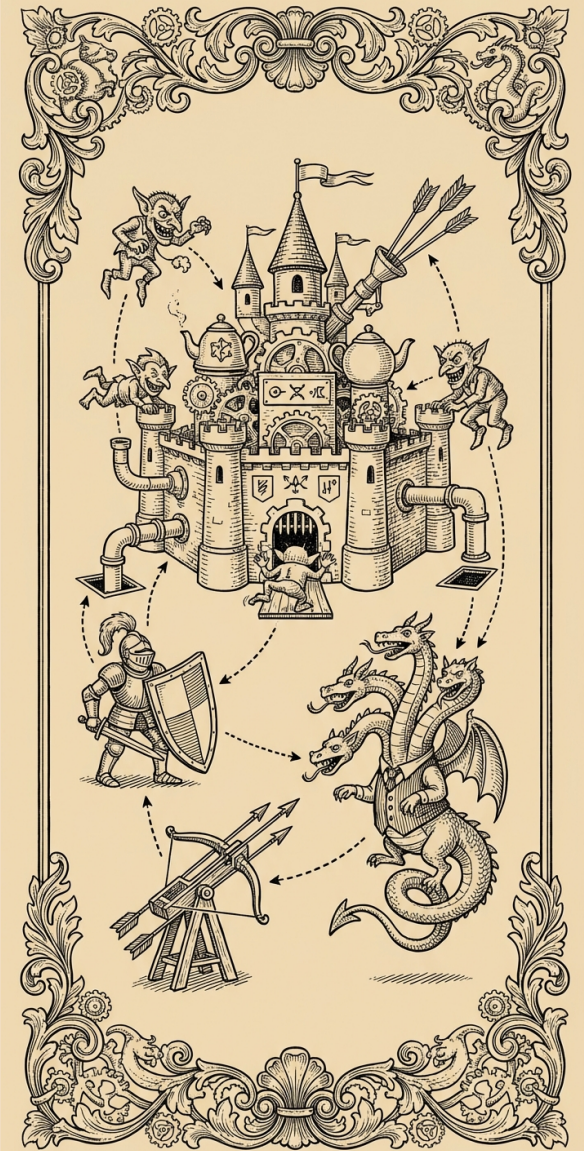
What capabilities and resources do they have?

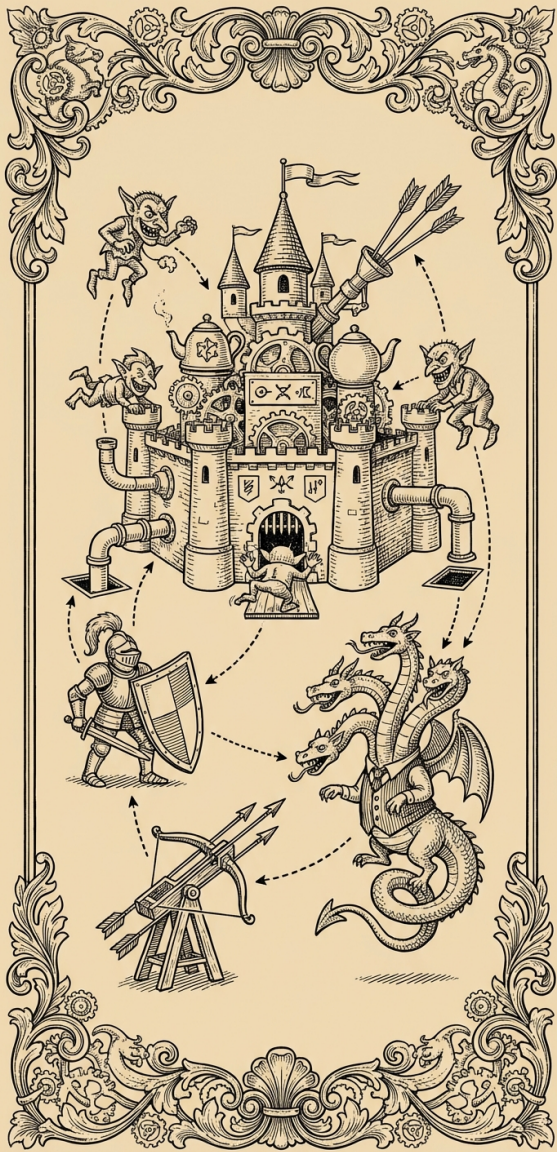
Consider the threat models implied by...

The lock on a bathroom door, in a house

The deadbolt at the front door of a house

A bank vault





Threat models in computer security include...

- Careless users who forget passwords and use USB drives they found in the parking lot
- “Script kiddies” downloading exploits
- Nation-states with nation-state budgets (if you’ve never read about Stuxnet, you should)

A CRYPTO NERD'S IMAGINATION:

HIS LAPTOP'S ENCRYPTED.
LET'S BUILD A MILLION-DOLLAR
CLUSTER TO CRACK IT.

BLAST! OUR
EVIL PLAN
IS FOILED!

NO GOOD! IT'S
4096-BIT RSA!



WHAT WOULD ACTUALLY HAPPEN:

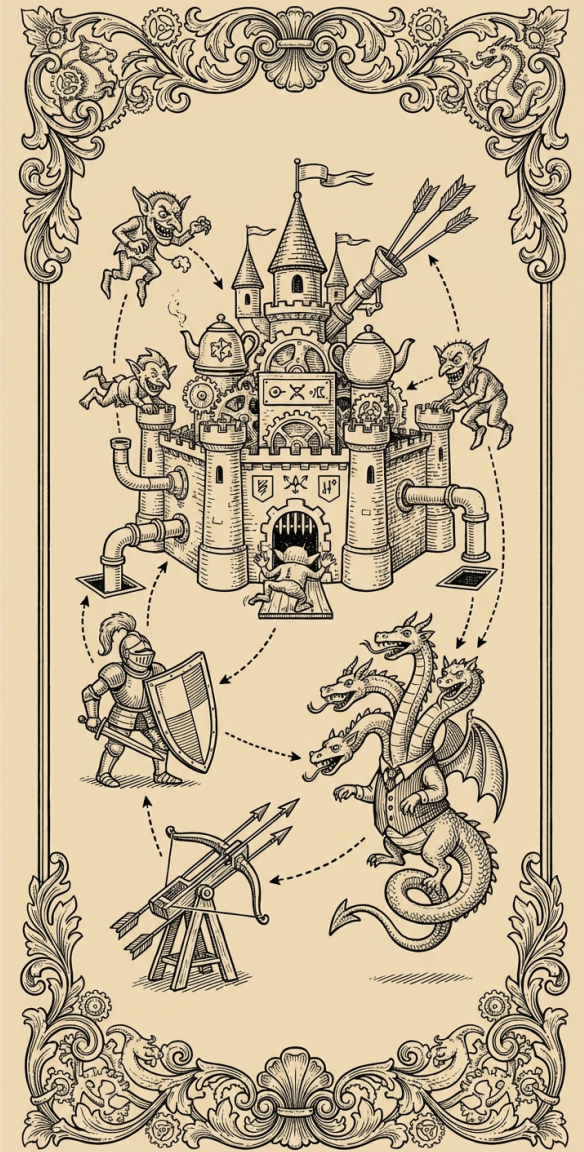
HIS LAPTOP'S ENCRYPTED.
DRUG HIM AND HIT HIM WITH
THIS \$5 WRENCH UNTIL
HE TELLS US THE PASSWORD.

GOT IT.



It's expensive or impossible to defend
against an attacker who is smart,
motivated, rich, and has a lot of time

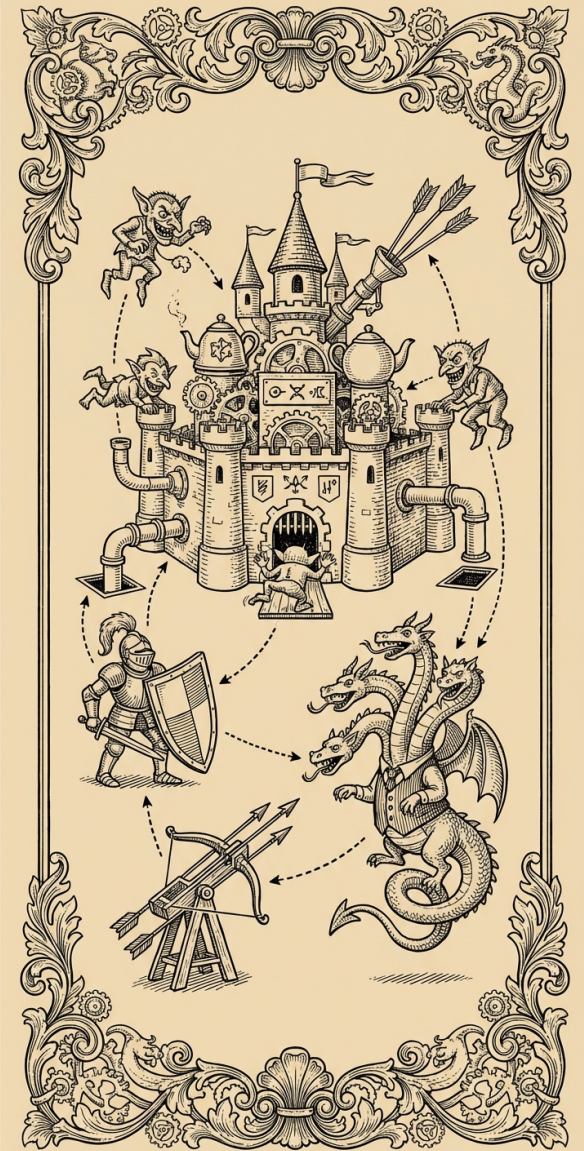
So generally our threat models are scoped
more modestly

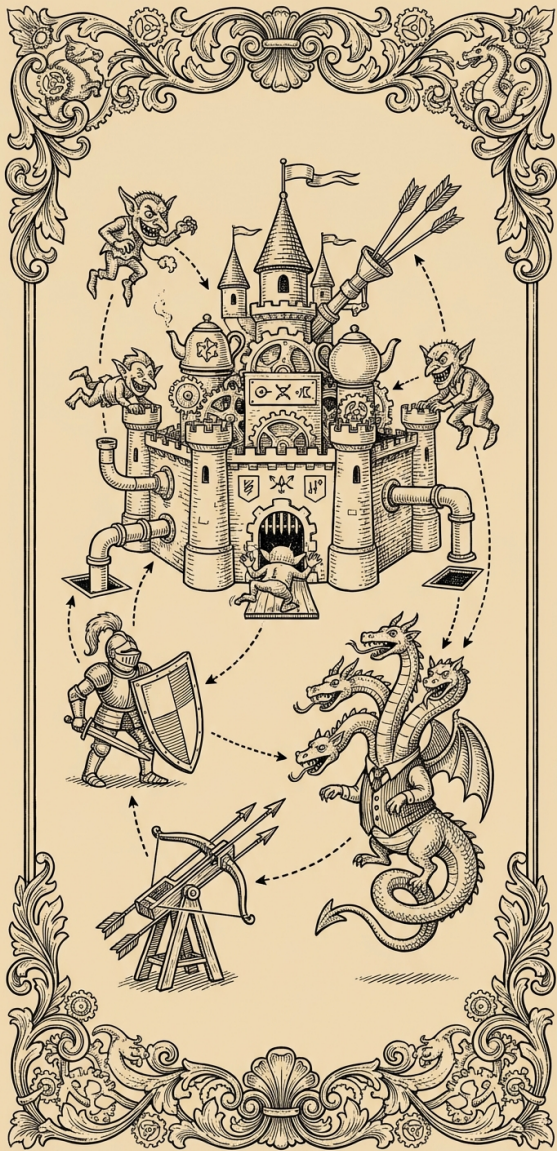


As with security policies: You can't expect an LLM to just understand your threat model

You'll have to write it down

Will the LLM listen? Unclear.





Some of the important questions:

Is our security policy a good one?

How does our code enforce the security policy?

Does our code correctly enforce the security policy?

What parts of the threat model are we defending against?

Are are our defenses commensurate with those threats?

Technical controls for security policies:

- Cryptography
- Access checks
- Type safety / memory safety
- Sandboxes
- Side channel avoidance





Correctly implementing / using these technical controls is extraordinarily difficult

Even hardened professionals cannot be trusted to avoid mistakes

Do LLMs make this worse?

Do Users Write More Insecure Code with AI Assistants?

Neil Perry*
Stanford University

Megha Srivastava*
Stanford University

Deepak Kumar
Stanford University / UC
San Diego

Dan Boneh
Stanford University

Overall, we find that participants who had access to an AI assistant wrote significantly less secure code than those without access to an assistant. Participants with access to an AI assistant were also more likely to believe they wrote secure code, suggesting that such tools may lead users to be overconfident about security flaws in their code.

(2023)

The Impact of AI-Assisted Development on Software Security: A Study of Gemini and Developer Experience

Nadine Jost
nadine.jost@rub.de
Ruhr University Bochum
Bochum, Germany

Benjamin Berens
benjamin.berens@kit.edu
Karlsruhe Institute of Technology
Karlsruhe, Germany

Manuel Karl
m.karl@tu-braunschweig.de
TU Braunschweig
Braunschweig, Germany

Stefan Albert Horstmann
stefan.horstmann@uni-koeln.de
University of Cologne
Cologne, Germany

Martin Johns
m.johns@tu-braunschweig.de
TU Braunschweig
Braunschweig, Germany

Alena Naiakshina
alena.naiakshina@uni-koeln.de
University of Cologne
Cologne, Germany

While we did not observe significant differences between using Gemini in terms of secure software development, programming experience significantly improved code security and cannot be fully substituted by Gemini.

(2026)

The Impact of AI-Assisted Development on Software Security: A Study of Gemini and Developer Experience

Nadine Jost
nadine.jost@rub.de
Ruhr University Bochum
Bochum, Germany

Stefan Albert Horstmann
stefan.horstmann@uni-koeln.de
University of Cologne
Cologne, Germany

Benjamin Berens
benjamin.berens@kit.edu
Karlsruhe Institute of Technology
Karlsruhe, Germany

Martin Johns
m.johns@tu-braunschweig.de
TU Braunschweig
Braunschweig, Germany

Manuel Karl
m.karl@tu-braunschweig.de
TU Braunschweig
Braunschweig, Germany

Alena Naiakshina
alena.naiakshina@uni-koeln.de
University of Cologne
Cologne, Germany

While we did not observe significant differences between using Gemini in terms of secure software development, programming experience significantly improved code security and cannot be fully substituted by Gemini.

(2024)



The jury is still out on the effect of LLMs on our ability to write secure code

The smart strategy would be to be very, very careful about LLM use in any security-sensitive environment



We can (and should) be clear which code enforces the security policy, vs code that does regular application logic

The second kind of code should be less susceptible to security bugs

But this separation only works if it exists. That is, if our system cleanly factors security code and application logic

In many cases this separation is not good

How do we get secure code in practice?

- When bugs get reported, we need to fix them
- Do lots of code audits
- Do lots of fuzzing
- Hire pentesters

All of these things take time and money, and there are no guarantees





So far we've been looking at the defensive side of computer security

The vast majority of development work is defensive in nature

But, attackers exist

And there's a fundamental asymmetry:
Defenders must block every attack, but
attackers only need to succeed once

LLMs are facilitating attacks

Recent ones are good at finding security bugs

Linux maintainer Greg Kroah-Hartman:

"Months ago, we were getting what we called 'AI slop,' AI-generated security reports that were obviously wrong or low quality," he said. "It was kind of funny. It didn't really worry us." Of course, there are many Linux kernel maintainers, so for them, AI slop isn't as burdensome as it is for, say, Daniel Stenberg, founder and lead developer of cURL, where AI slop reports caused the cURL team to stop paying bug bounties.

Things have changed, Kroah-Hartman said. "Something happened a month ago, and the world switched. Now we have real reports." It's not just Linux, he continued. "All open source projects have real reports that are made with AI, but they're good, and they're real." Security teams across major open source projects talk informally and frequently, he noted, and everyone is seeing the same shift. "All open source security teams are hitting this right now."





Things have recently changed and we don't know yet where this is going. However...

- Defenders are overwhelmed
- The current situation favors attackers
- Software projects that can devote a lot of resources to fixing bugs might come out ahead, a few years from now
- Other projects are in trouble

A lot of recent think pieces are bemoaning the fact that LLMs make computer security into a pay to play activity

But computer security has always been pay to play in the sense that both the offensive and defensive sides are expensive to do well

Hiring smart people, running big fuzzing campaigns, etc. — none of this was ever cheap



Anthropic claims their latest model, “Claude Mythos,” is a game changer

<https://red.anthropic.com/2026/mythos-preview/>

<https://www-cdn.anthropic.com/08ab9158070959f88f296514c21b7facce6f52bc.pdf>

But Anthropic is also anticipating a
\$380,000,000,000 IPO

So do we want to believe them?





Implications for you:

To the extent that LLMs allow you to outsource thinking, they will make it less likely that you'll write secure code

This is an area where training and experience are incredibly valuable

Also, LLMs seem to be favoring attackers for now

But, we think there are some upsides as well